

Kalshi Prediction Market Inefficiency Investigation

Colby Whitehouse, Jack Decker, Will Sullivan

Summary

This project set out to identify and exploit pricing inefficiencies on Kalshi, a prediction market where retail participation, thin liquidity, and related contracts can cause prices to drift out of sync. The original proposal and presentation focused on three main theses: fixed-outcome basket arbitrage, Bitcoin range-versus-threshold parity, and short-duration Bitcoin pricing. After reviewing the full repository, the project should now be described more broadly. In addition to those three strategies, the codebase includes general market data tools, an automated basket scanner, a culture and entertainment market flow analyzer, and Spotify chart signal research.

Our strongest result still came from Strategy #1, the basket arbitrage approach. This is the most complete part of the codebase and the clearest live-trading success. The existing report recorded +\$5.86 of profit across three live-traded markets, with a weighted overall ROI of +17.2%. That result is still included here as the team's documented live-trading result, but the repository does not include a complete fill-by-fill trade ledger that would let the number be independently recomputed from raw exports.

Strategy #2, the BTC range-versus-threshold parity strategy, was valuable mostly as a negative result. The math and parsing code are substantial, and the local tests pass, but live markets generally kept those related books aligned when liquidity was high enough to trade safely. Strategy #3 split into a 15-minute crypto probability model and a spike-driven latency prototype. Both are promising research directions, but the code shows them as experimental live scripts rather than production systems.

The biggest update from the original report is that the repository now includes two additional research tracks. Strategy #4 is an installable Kalshi culture/event flow analyzer that collects snapshots and trades into SQLite, scores aggressive YES buying, resolves outcomes, replays scores, and exports reports. Strategy #5 is a Spotify chart data pipeline that collects and aggregates artist-stream features. These are not completed live trading strategies yet, but they are meaningful parts of the project and should be reflected in the final report.

Strategies and Results

Strategy #1 - Basket Arbitrage on Fixed-Outcome Events

The core idea behind Strategy #1 was that in an event where exactly one outcome, or a known fixed number of outcomes, must occur, buying a complete covered basket can guarantee a fixed payout. If the total cost of the basket, including fees and execution costs, is below that payout, the trade has positive edge in theory. The practical problem is that the arbitrage only stays safe if all legs fill at the expected prices and if fees, slippage, and partial-fill risk are handled correctly.

The implementation reflects that practical focus. `basket.py` defines the fee model and basket evaluation logic, including depth-aware fill sizing, cost, payout, edge, and ROI. `normalize.py` and

kalshi_fixed_point.py normalize Kalshi orderbooks, parse fixed-point dollar fields, infer asks from opposite-side bids, and handle price and quantity formatting. client.py and kalshi_common.py provide REST and websocket authentication helpers. The tests in tests/test_basket.py and tests/test_normalize.py cover the deterministic basket math and orderbook normalization behavior. The manual analyzers are more extensive than the original report described. analyze_event.py and analyze_event_arb.py handle event-level analysis, orderbook parsing, maker and midpoint limit planning, trade-state synchronization, pending order awareness, fill summaries, price warnings, and basket reporting. analyze_universe.py and market_finder.py expand the search across many events and use close-time, projected-time, and liquidity filters to find candidates. place_limit_orders.py converts analyzer output into executable limit-order batches.

The key live-trading lesson was that theoretical arbitrage is not automatically risk-free. Crossing the spread makes it easier to complete every leg, but taker fees and slippage can eliminate the edge. Resting one tick above the best bid or at the midpoint can improve economics and avoid some fees, but it creates fill risk and adverse-selection risk. This is why the project moved toward hybrid limit-order logic rather than simple spread crossing.

This remained the strongest demonstrated edge. The prior report documented profitable trades in a Freestyle Skiing medal basket and Trump Truth Social range baskets. The codebase also contains saved Olympics event data that illustrates why multi-winner events matter: medal markets can have three winners, so a scanner that assumes exactly one YES outcome will skip opportunities that can still be profitable.

Strategy #1(s) - Automated Market Scanner

After proving the basket logic manually, the project extended it into an automated scanner. full_set_arb.py is the main scanner engine. It can load or fetch event data and orderbooks, filter tradeable markets, walk YES and NO orderbook ladders, apply Kalshi fee formulas and cached series fee multipliers, compute guaranteed YES and NO payouts, and size positions under a budget cap. scan_closing_arb.py prioritizes events close to expiration, where thin liquidity and retail behavior may create more mispricings.

The scanner also includes resolution inference that was not fully captured in the original report. resolution.py can use explicit overrides from resolution_overrides.json, Kalshi's mutually_exclusive flag, top-N language, and medal-event wording to infer how many markets should resolve YES. The scanner tests cover mutually exclusive events, F1 top-5 and top-10 style overrides, medal events, and partial-payout logic.

The scanner is useful, but it is not finished as an execution system. It still needs tighter integration with the hybrid limit-order logic from the manual analyzer, active order management, and more robust result accounting. The correct conclusion is that the scanner can find and evaluate candidate opportunities, but it should not yet be treated as a fully automated trading bot.

Strategy #2 - BTC Range vs. Threshold Parity

Strategy #2 focused on parity relationships between Kalshi Bitcoin range markets and threshold markets. Since both describe the same underlying BTC distribution at the same expiration time, their prices should imply consistent probabilities. A threshold probability should equal the sum of all

higher range buckets, and a range bucket should equal the difference between neighboring threshold probabilities. If those relationships break by more than fees and spreads, a covered trade can theoretically lock in profit.

The codebase implements this framework in detail. `kxbtc_parity_parse.py` classifies markets as ranges or thresholds. `kxbtc_parity_math.py` defines buckets, thresholds, trade legs, trade recipes, bucket-sum checks, monotonic threshold checks, negative implied bucket checks, threshold-from-bucket trades, bucket-from-threshold trades, box trades, and constant-payout trades. `analyze_kxbtc_parity.py` pulls live books and estimates executable edge after costs. `kxbtc_parity_maker_bot.py` explores passive maker prices that could create positive edge. The tests in `tests/test_kxbtc_parity.py` cover the parsing and parity math.

The practical finding was that this market relationship was usually too efficient to exploit safely. During active weekday periods, prices across the related BTC books generally stayed aligned. During weekends or lower-volume periods, apparent discrepancies appeared more often, but spreads widened, fills became unreliable, and passive orders created adverse-selection risk. This made Strategy #2 a useful negative result: the math was sound, but the live opportunity was much smaller than expected.

Strategy #3A - 15-Minute Crypto Probability Model

Strategy #3A moved from static arbitrage to fair-value modeling. The idea was to estimate the probability that a 15-minute crypto contract would resolve YES, compare that probability to Kalshi's implied price, and trade only when the difference was large enough to overcome fees and risk. The main implementation is `15min_scanner.py`, with an earlier version in `scanner.py` and order/authentication helpers in `test_order.py`.

The model builds current 15-minute event tickers for BTC, ETH, XRP, and SOL, fetches Coinbase spot prices and recent one-minute candles, estimates volatility from recent log returns using an exponentially weighted variance, and compares model probabilities to live Kalshi quotes. The scanner also checks existing position exposure and sizes orders by max risk dollars rather than a fixed contract count, which is a more sensible approach when contract prices vary widely.

The model looked promising in limited live testing, especially when near-side contracts traded around 20 to 30 cents. That is where market participants may price tails less accurately. However, the repository does not contain enough logged trades or backtest output to prove durability. This strategy should be described as promising but inconclusive, and it needs cleaner packaging, dependency management, and a longer out-of-sample trade record before scaling.

Strategy #3B - BTC Spike HFT

Strategy #3B emerged from watching the 15-minute crypto contracts in real time. The team noticed that Kalshi contract prices could lag sudden moves in the underlying BTC market, partly because the settlement price is based on a moving average. `PriceHFT.py` is the prototype built around that observation.

The script subscribes to live Coinbase trade data, keeps a rolling history of recent BTC prices, detects short-window spikes, scales the signal by time remaining in the 15-minute window and an assumed variance, estimates the expected Kalshi contract move, and attempts to enter YES or NO

before exiting shortly afterward. The file also contains Binance feed code, though the current main routine gathers the Coinbase feed.

This is the most infrastructure-sensitive strategy. The repo shows it as a prototype, not a hardened high-frequency trading system. It still needs deployment close to the exchange, better parameter tuning, cleaner ticker construction, position and order-risk controls, and more robust exit logic. The right conclusion is that the lag appeared real, but capturing it reliably requires faster infrastructure and more testing than the team completed during the semester.

Market Data Tools

MarketDataTools is supporting infrastructure rather than a separate trading thesis. `track.py` connects to Kalshi websocket channels and records best bid/offer and trade events to CSV logs. `kalshi_bbo_one_market.py` is a smaller one-market watcher. `kalshi_common.py` centralizes websocket signing and orderbook state handling. `analyze.py` reads the CSV logs and computes diagnostics such as spreads, midpoint movement, trade bursts, weighted medians, and volume proxies.

These files matter because they show the project was not just a set of one-off trading scripts. The team built tools for collecting and analyzing market microstructure data, which is necessary for evaluating whether an apparent edge is real.

Strategy #4 - Culture/Event Flow Analyzer

Strategy #4 is a newer addition that was missing from the original report. It is an installable Python package called `kalshi-culture-analyzer`, with a `pyproject.toml`, `README`, design spec, `YAML` configuration example, `CLI`, package modules, and tests. The goal is to analyze culture and entertainment Kalshi markets, such as reality TV or awards markets, where aggressive YES buying may reveal informed flow before resolution.

The `CLI` supports `collect`, `report`, `resolve`, `replay`, and `export` workflows. `collector.py` expands event tickers into markets, fetches snapshots and trades, infers trades from volume changes when needed, computes features, and writes everything to `SQLite`. `storage.py` defines tables for events, markets, snapshots, trades, features, scores, outcomes, and collector state. `features.py` computes aggressive-buying metrics, spread, volume, open interest, mid-price changes, and favorite status. `rules.py` scores markets using configurable weights for aggressive bursts, large aggressive trades, step changes, and sustained favorites.

This is a real analytical system, but it is not yet a proven trading strategy. The repository does not include a populated `SQLite` database or post-resolution accuracy study. The next step is to collect real culture-event data, resolve outcomes, replay the scores, and test whether high-score markets predict YES resolutions better than the market-implied baseline.

Strategy #5 - Spotify Chart Signal Research

Strategy #5 is another newer addition. It explores Spotify chart data as an external signal source, likely for culture or music-related Kalshi markets. `get_charts.py` fetches U.S. daily chart CSVs. `artist_weekly.py` aggregates artist stream totals over a week. `artist_weekly_from_csvs.py` works

from saved CSV files, splits credited and featured artists, applies fractional credit by default, and computes total streams, rank-weighted streams, unique tracks, and number of chart days. The repository includes 13 saved daily U.S. chart CSVs from January 30 through February 5 and February 20 through February 25, 2026, plus aggregate artist-output CSVs. The local data shows high raw stream totals for artists such as Don Toliver, Bad Bunny, Olivia Dean, Ella Langley, and Bruno Mars. However, this pipeline is not wired into a Kalshi market parser, scanner, or execution workflow. It should be described as exploratory signal research, not as a tested Kalshi trading system.

Challenges, Pivots, & Lessons Learned

The biggest lesson from the project was that a guaranteed payout structure is only guaranteed after every leg is filled. Before that point, partial fills, price movement, taker fees, and adverse selection can turn a clean-looking arbitrage into unwanted directional exposure. This forced the project to move from simple top-of-book math toward depth-aware sizing, fee modeling, pending-order awareness, and hybrid limit-order execution.

A second major lesson was that event resolution structure matters. A one-winner event, a top-5 event, a top-10 event, and a medal event all require different basket payout assumptions. The scanner originally missed exactly the kind of multi-winner opportunity that proved important in live trading. The current resolution inference code and override file address this, but the logic still needs to be integrated more tightly with scanner execution.

The Strategy #2 pivot was also important. The team initially expected BTC range-versus-threshold parity to be a major arbitrage source. Instead, active markets were usually efficient, and thin markets were hard to trade safely. That negative result pushed the project toward areas where retail behavior, latency, or external information might matter more.

The later Strategy #4 and Strategy #5 work show that pivot. Rather than only looking for static arbitrage, the codebase began exploring signal discovery: aggressive YES flow in culture markets and external audience data from Spotify charts. These additions broaden the project, but they also create a need for better integration and more disciplined result tracking.

From an engineering standpoint, the repository also shows the value of normalizing data early, caching stable API inputs, keeping deterministic math under tests, and separating research prototypes from production-ready tools. Some folders are well-tested modules, while others are experimental live scripts with hardcoded paths or unpinned dependencies. That distinction should be clear in any final description of the project.

Validation and Code Status

The repository includes local tests for the most deterministic parts of the project. Because pytest was not installed in the bundled runtime used for this update, the tests were run with a lightweight import-and-call runner instead. Basket arbitrage had six passing tests covering basket math and normalization. The scanner had nine passing tests covering resolution inference and guaranteed payout logic. The BTC parity strategy had ten passing tests covering parsing and parity math. Strategy #4 had three passing tests covering ticker parsing and scoring rules, although its normal import path expects PyYAML, which was not installed in the bundled runtime.

Those tests are useful but incomplete. They do not cover authenticated Kalshi API calls, live websocket feeds, actual order placement, Coinbase or Binance network dependencies, SQLite collection at scale, Spotify fetching against the network, or full end-to-end profitability. The project should therefore be described as a mixture of tested core math, working analysis tools, and experimental live prototypes.

Next Steps

The clearest next step is to unify the Strategy #1 scanner and execution workflow. The scanner should support multi-winner events, use the same hybrid limit-order logic as the manual analyzer, and actively manage partially filled baskets. It should move resting orders as the book changes, cap exposure, and cross the spread only when necessary to complete a guaranteed basket.

The second priority is result accounting. The project needs a trade ledger that records every order, fill, fee, position, and settlement. Without that, reported ROI must be treated as manually documented performance rather than something reproducible from project artifacts.

For Strategy #3A and Strategy #3B, the next steps are more data and better infrastructure. The probability model needs weeks of logged trades and cleaner dependency packaging. The spike strategy needs deployment closer to the exchange, parameter tuning, and stronger order-risk controls before it can be judged fairly.

For Strategy #4, the next step is a real backtest. The collector should run over selected culture events, outcomes should be resolved after market close, and replay should measure whether the aggressive-flow score predicts outcomes better than market-implied probabilities. For Strategy #5, the next step is to connect Spotify artist features to actual tradeable Kalshi markets or keep the work clearly labeled as exploratory data research.

Finally, the report artifacts should be kept inside the repository. The 4-Report folder currently contains only .gitkeep, while the active DOCX lives in Downloads. Moving the final report into 4-Report would make the repository easier to review and keep synchronized.

Conclusion

Overall, this project showed that Kalshi does contain real inefficiencies, but the value of a strategy depends heavily on execution, market structure, data quality, and infrastructure. Basket arbitrage on fixed-outcome events remains the strongest proven edge in the repository. BTC range-versus-threshold parity was a useful negative result because it showed that some theoretically attractive relationships are already efficiently priced. The short-duration crypto strategies are promising but still experimental. The culture-flow analyzer and Spotify chart tools broaden the project into signal discovery, but they still need outcome-based validation.

The most accurate summary is that the project evolved from a simple arbitrage search into a broader trading research system. It now includes fee-aware basket math, market scanners, orderbook normalization, websocket logging, parity analysis, crypto modeling, latency prototypes, culture-market flow scoring, and external chart-data feature engineering. The next phase should focus less on adding isolated scripts and more on connecting these pieces into one reproducible pipeline for data collection, signal generation, execution, risk management, and performance measurement.